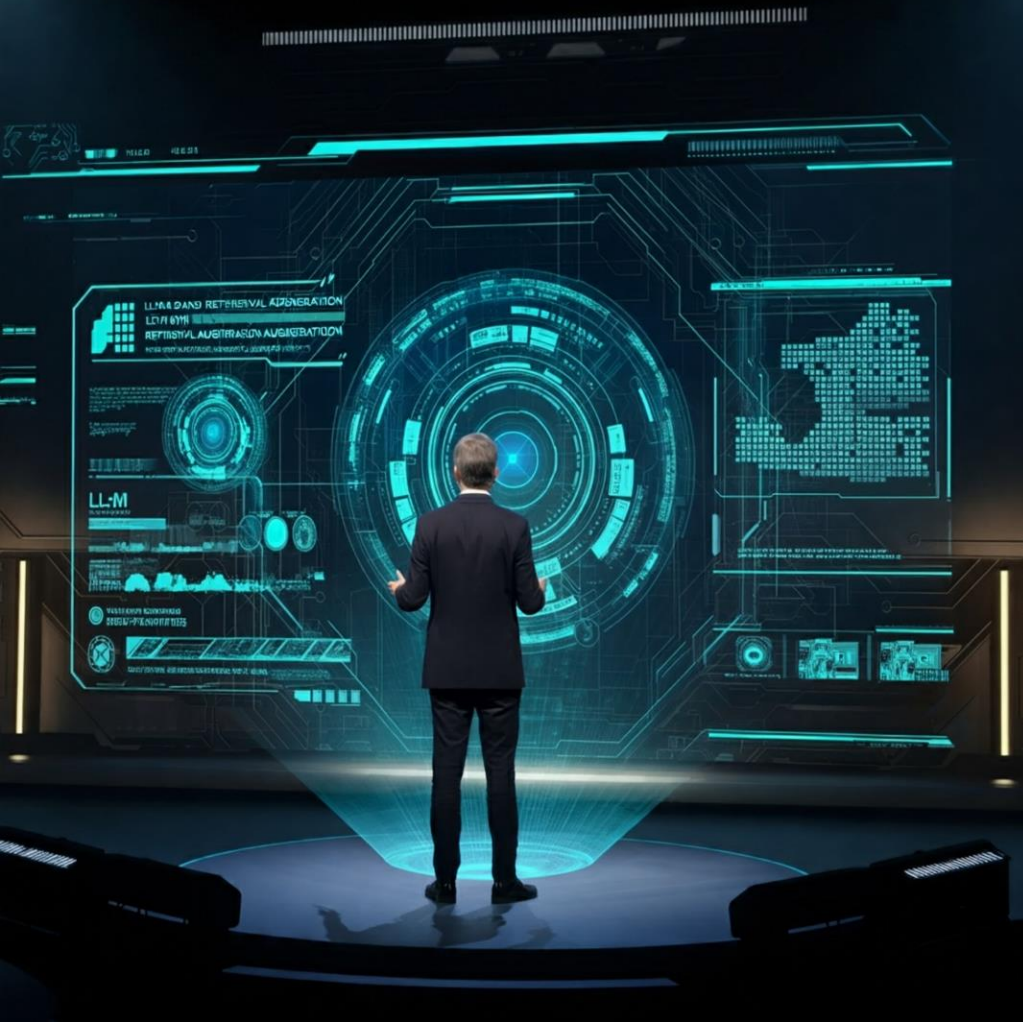




LLM, RAG and Building AI Agents – Part 1

Onkar Telang
3 Oct 2024

“AI Is The First Technology That Has No Limit” - Bill Gates, Co-Founder
Microsoft

AI

"AI is one of the most profound things we're working on as humanity. It's more profound than fire or electricity." - Sundar Pichai, CEO Alphabet

“The hottest new programming language is English”
- Andrej Karpathy, Stanford Graduate, Former AI research group, OpenAI

”

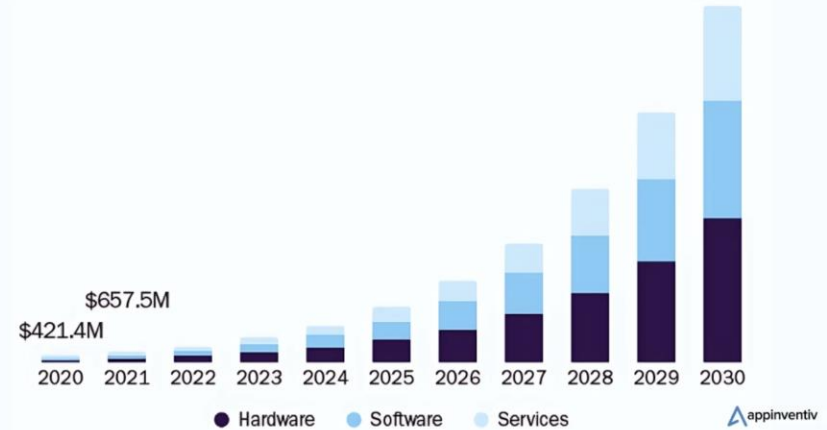
Table of Content

LLM for Software Engineering & Industry
Artificial Intelligence & Machine Learning
Deep Learning
Language Model – Gen AI
Tokenization
Word Embedding
Deep Dive In – Transformer
Bert
GPT
Fine Tuning
LLM as a Transformer

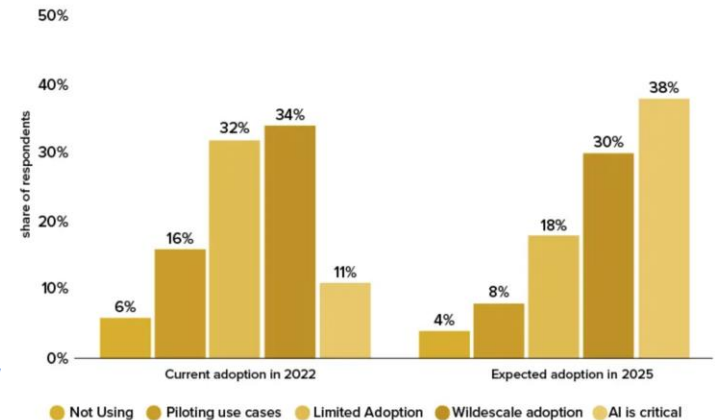
LLM Growth



Asia Pacific Autonomous AI & Autonomous Agents Market



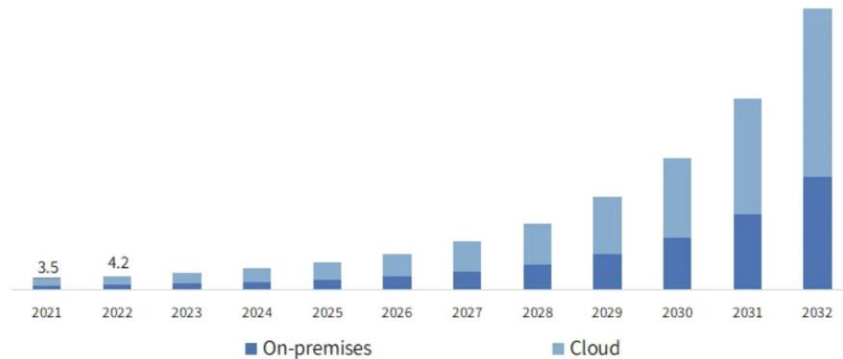
AI Adoption Rate in Supply Chain Globally: 2022- 2025



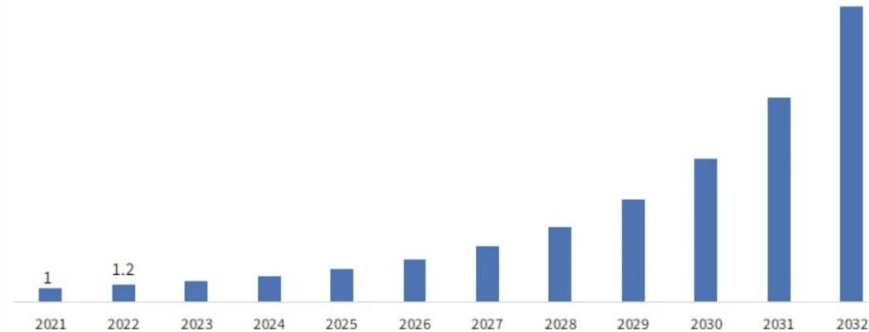
<https://appinventiv.com/blog/ai-in-supply-chain-analytics/>

LLM Industrial Growth

Autonomous AI and Autonomous Agents Market Size, By Deployment Model, 2021 – 2032, (USD Billion)

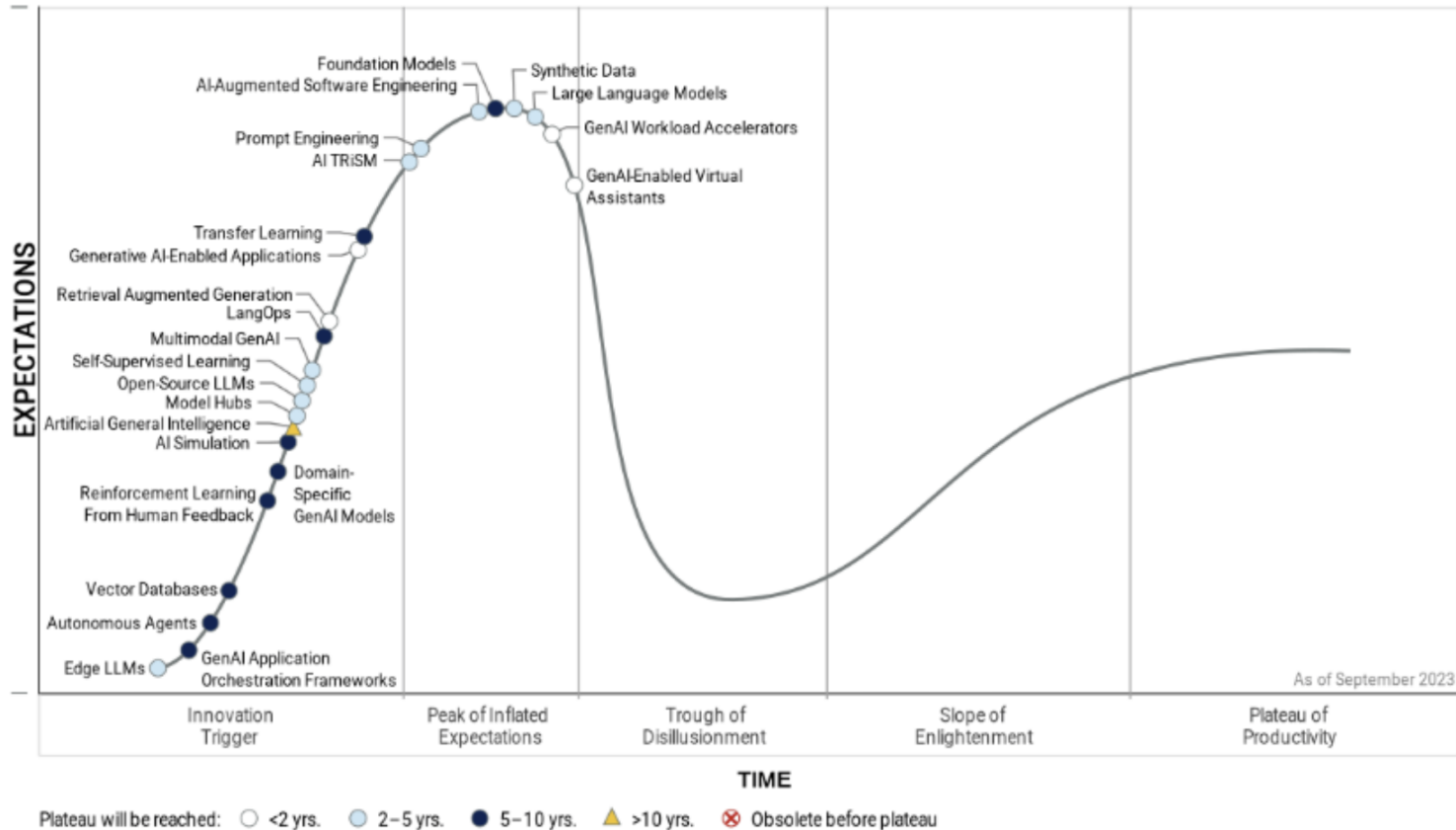


U.S. Autonomous AI and Autonomous Agents Market, 2021 -2032, (USD Billion)



<https://www.gminsights.com/industry-analysis/autonomous-ai-and-autonomous-agents-market>

LLM for Software Engineering



LLM for Software Engineering

From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future

Haolin Jin, Linghan Huang, Haipeng Cai, Jun Yan, Bo Li, Huaming Chen

Abstract—With the rise of large language models (LLMs), researchers are increasingly exploring their applications in various vertical domains, such as software engineering. LLMs have achieved remarkable success in areas including code generation and vulnerability detection. However, they also exhibit numerous limitations and shortcomings. LLM-based agents, a novel technology with the potential for Artificial General Intelligence (AGI), combine LLMs as the core for decision-making and action-taking, addressing some of the inherent limitations of LLMs such as lack of autonomy and self-improvement. Despite numerous studies and surveys exploring the possibility of using LLMs in software engineering, it lacks a clear distinction between LLMs and LLM-based agents. It is still in its early stage for a unified standard and benchmarking to qualify an LLM solution as an LLM-based agent in its domain. In this survey, we broadly investigate the current practice and solutions for LLMs and LLM-based agents for software engineering. In particular we summarise six key topics: requirement engineering, code generation, autonomous decision-making, software design, test generation, and software maintenance. We review and differentiate the work of LLMs and LLM-based agents from these six topics, examining their differences and similarities in tasks, benchmarks, and evaluation metrics. Finally, we discuss the models and benchmarks used, providing a comprehensive analysis of their applications and effectiveness in software engineering. We anticipate this work will shed some lights on pushing the boundaries of LLM-based agents in software engineering for future research.

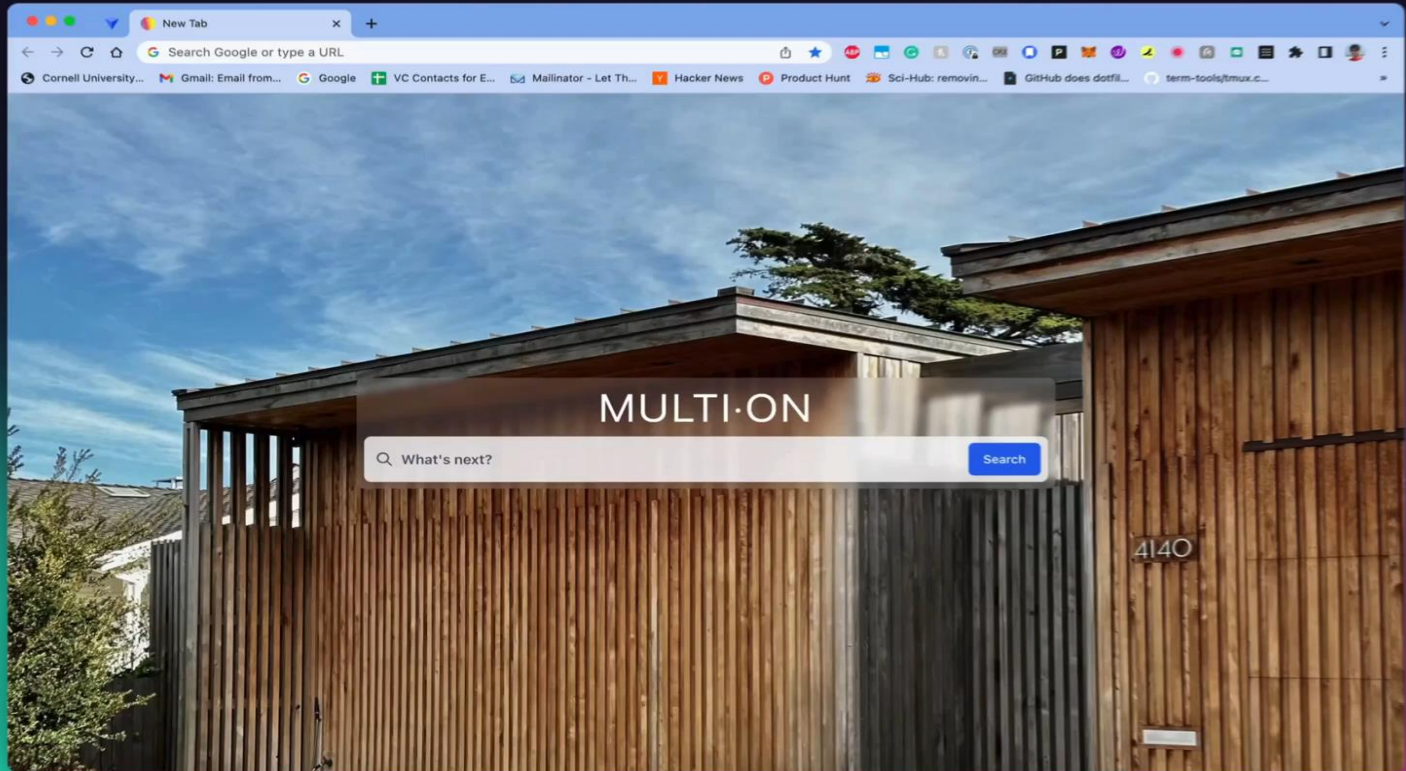
Index Terms—Large Language Models, LLM-based Agents, Software Engineering, Benchmark, Software Security, AI System Development

as GPT [3] and Codex [4], have demonstrated remarkable capabilities in handling downstream tasks in SE, including code generation, debugging, and documentation. These models leverage vast amounts of training data to generate human-like text, offering unprecedented levels of fluency and coherence. Studies have shown that LLMs can enhance productivity in software projects by providing intelligent code suggestions, automating repetitive tasks, even generating entire code snippets from natural language descriptions [5].

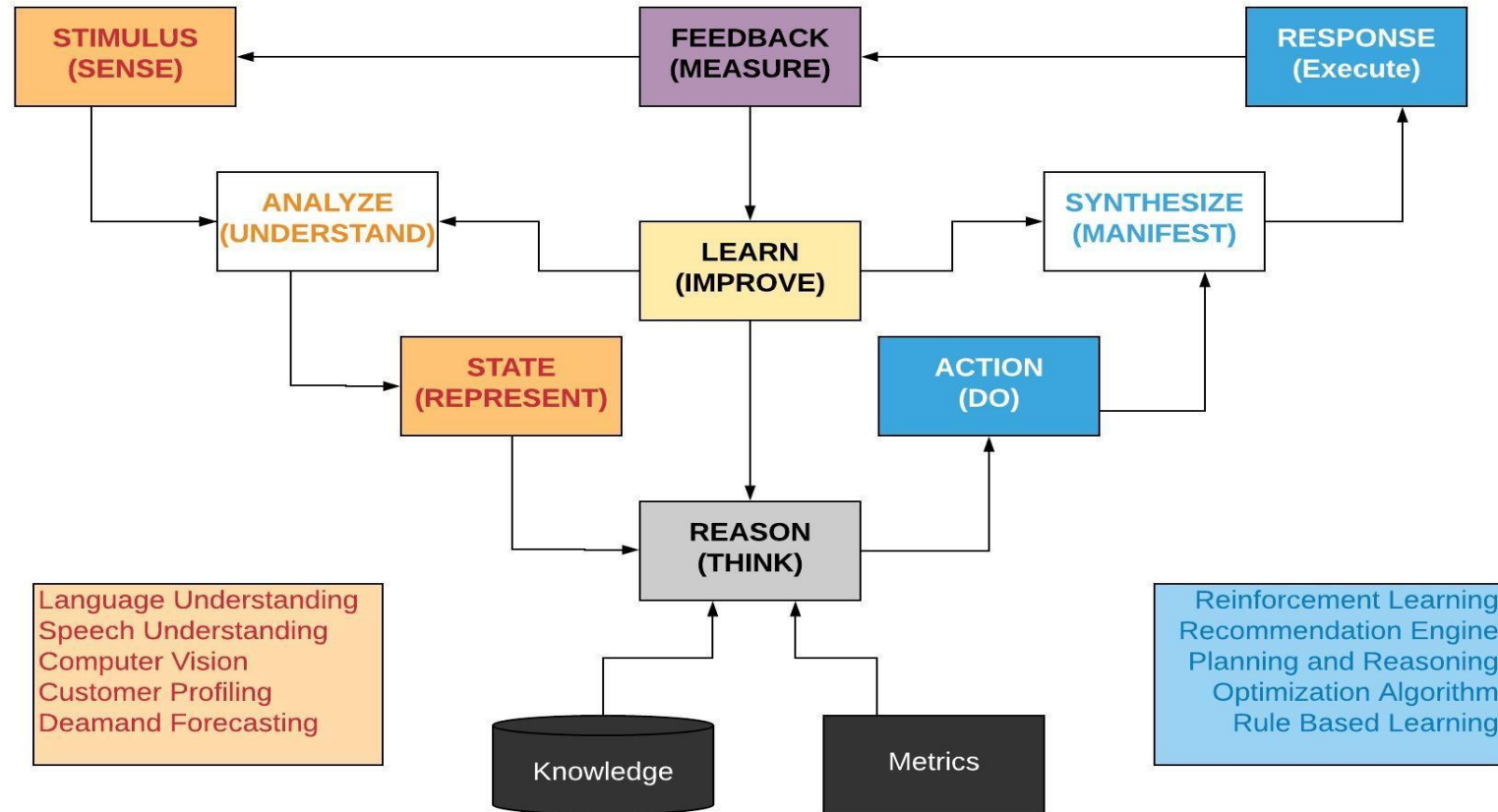
Despite their potential, there are significant challenges in applying LLMs to SE. One major issue is their limited context length [6], which restricts the model's ability to comprehend and manage extensive codebases, making it challenging to maintain coherence over prolonged interactions. Hallucinations is another main concern, where the model generates code that appears plausible but is actually incorrect or nonsensical [7], potentially introducing bugs or vulnerabilities if not carefully reviewed by experienced developers. Additionally, the inability of LLMs to use external tools restricts their access to real-time data and prevent them from performing tasks outside their training scope. It diminishes their effectiveness in dynamic environments. These limitations significantly impact the application of LLMs in SE, and also highlight the need for expert developers to critically refine and validate LLM-generated code for accuracy and security [8]. In complex projects, the static nature of LLMs can hinder their ability

Category	LLMs	LLM-based agents
Requirement Engineering and Documentation	Requirement Classification and Extraction (3) Requirement Generation and Description (4) Requirements Satisfaction Assessment (1) Specification Generation (3) Quality Evaluation (2) Ambiguity Detection (2)	Generation of Semi-structured Documents (1) Generate safety requirements (1) Automatically generating use cases based on high-level requirements (1) Automated User Story Quality Enhancement (2)
Code Generation and software development	Code Generation Debugging (3) Code Evaluation (2) Implement HTTP server (1) Enhancing Code Generation Capabilities (3) Specialized Code Generation (2) Human Feedback Preference Simulation (1)	Automating the Software Development Process (5) Large-Scale Code and Document Generation (1) Tool and External API Usage (2) Multi-Agent Collaboration and Code Refine (2) Improving Code Generation Quality (2)
Autonomous Learning and Decision Making	Multi-LLMs Decision-Making (1) Creativity Evaluation (1) Self-Identify and Correct Code (1) Judge Chatbot Response (1) Mimics Human Scientific Debugging (1) Deliberate Problem Solving(1)	Collaborative Decision-Making and Multi-Agent Systems (5) Autonomous Reasoning and Decision-Making (7) Learning and Adaptation Through Feedback (4) Simulation and Evaluation of Human-like Behaviors (2)
Software Design and Evaluation	Creative Capabilities Evaluation (1) Performance in SE Tasks (1) Educational Utility and Assessment (1) Efficiency Optimization (2)	Automation of Software Engineering Processes (3) Enhancing Problem Solving and Reasoning (4) Integration and Management of AI Models and Tools (3) Optimization and Efficiency Improvement (2) Performance Assessment in Dynamic Environments (2)
Software Test Generation	Bug Reproduction and Debugging (2) Security Test (2) Test Coverage (2) Universal Fuzzing (1)	Multi-agent Collaborative Test Generation (2) Autonomous Testing and Conversational Interfaces (3)
Software Security & Maintainance	Vulnerability Detection (6) Vulnerability Repair (2) Program Repair (4) Robustness Testing (1) Requirements Analysis (1) Fuzzing (1) Duplicate Entry (1) Code Generation and Debugging (4) Penetration Testing and Security Assessment (2) Program Analysis and Debugging (1)	Autonomous Software Development and Maintenance (4) Debugging and Fault Localization (4) Vulnerability Detection and Penetration Testing (3) Smart Contract Auditing and Repair (2) Safety and Risk Analysis (2) Adaptive and Communicative Agents (1)

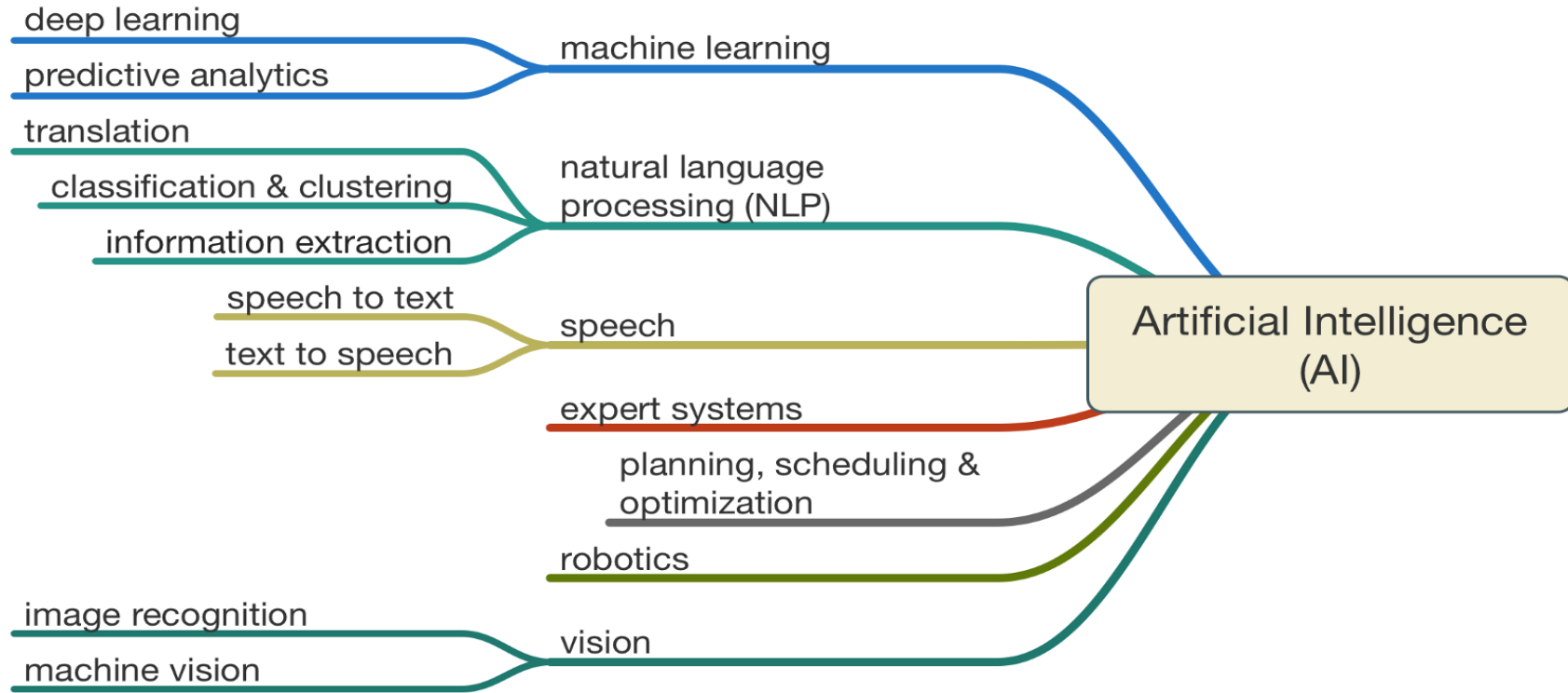
Flight Booked By AI



Artificial + Intelligence



AI is an Umbrella Term

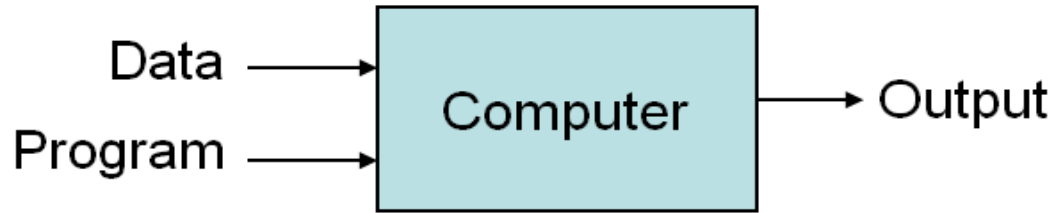


Machine Learning

Machine Learning

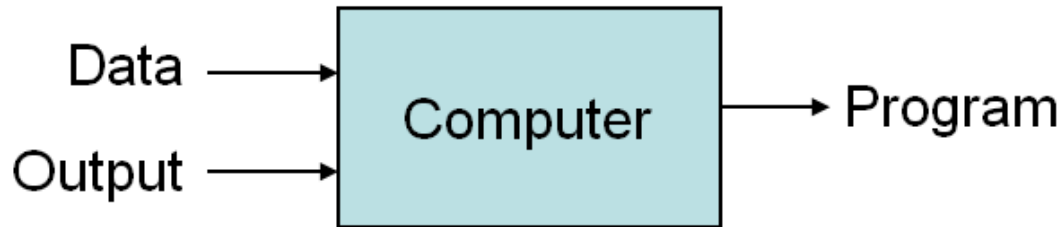
Way of Achieving AI → A Subfield of it

Traditional Programming



Computer Learn to Program
themselves

Machine Learning



Machine Learning Approach

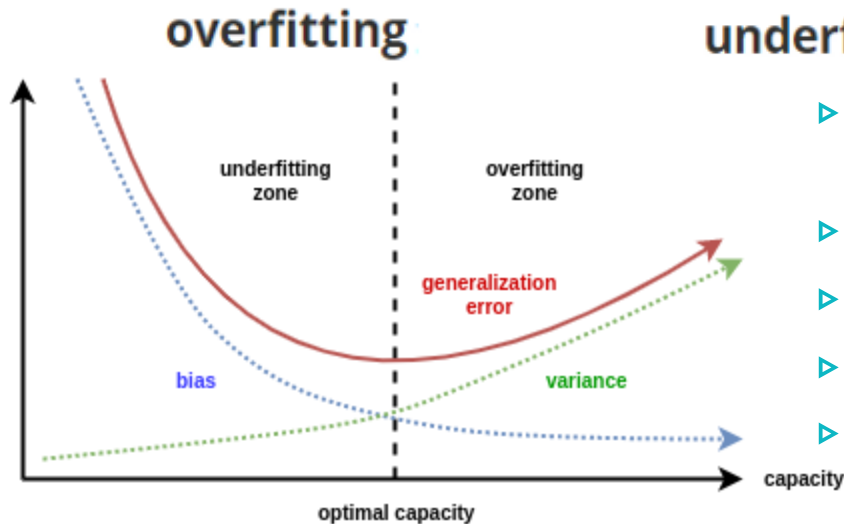
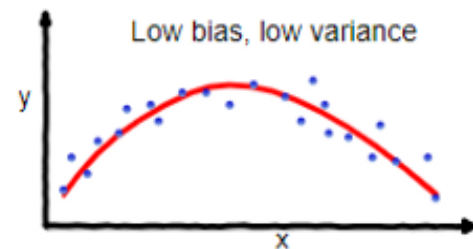
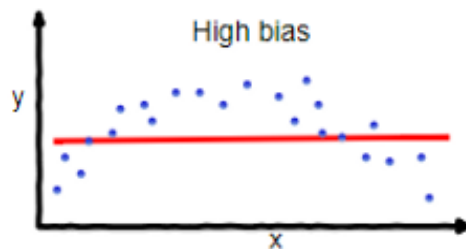
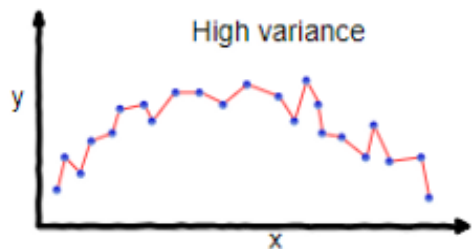
- ▶ Computers ability to learn without being explicitly program **Learn from Data** →
- ▶ Algorithms that do the learning **without human intervention**.
- ▶ System able to make **Prediction** or **Take Decision** from **Past Data**
- ▶ **Goal: (From Machine Learning Models)**

Learning function $f: x \rightarrow y$ to make correct prediction for **New Input Data**

- Choose function (Hypothesis) family (e.g logistic regression, SVM)
- Optimize parameters on training data: Minimize Loss $\sum (f(x) - y)^2$



Issue with Machine Learning



- ▶ Cross Validation & Regularization are ways to avoid Over-fitting
- ▶ More Training examples → Fix high Variance
- ▶ Smaller Sets of Feature → Fix high Variance
- ▶ Get additional Feature → Fix high Bias
- ▶ Increasing λ → Fix high Variance

Deep Learning & ANN

“Computers will understand sarcasm before Americans do.” ~ Geoffrey Hinton

Artificial Neural Network & Deep Learning

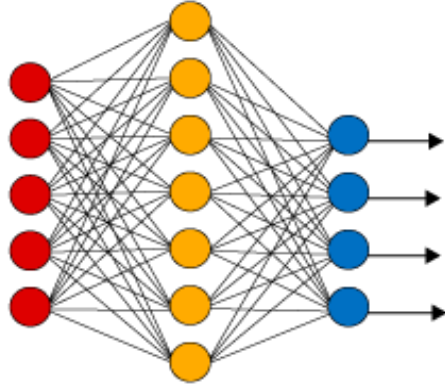


From Google I/O 2018 Keynote Speaker Sundar Pichai

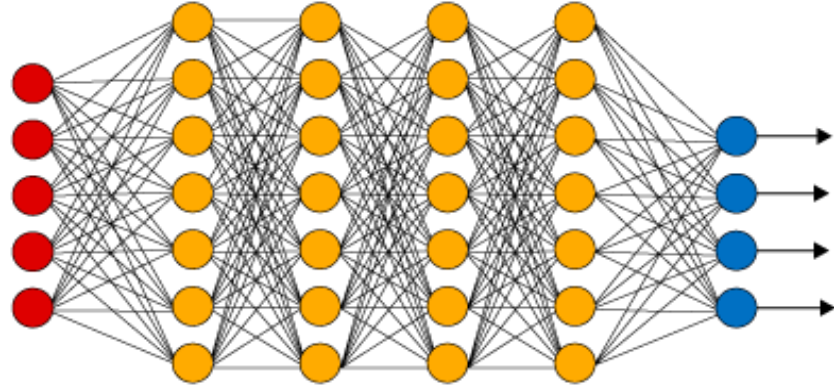


Artificial Neural Network & Deep Learning

Simple Neural Network



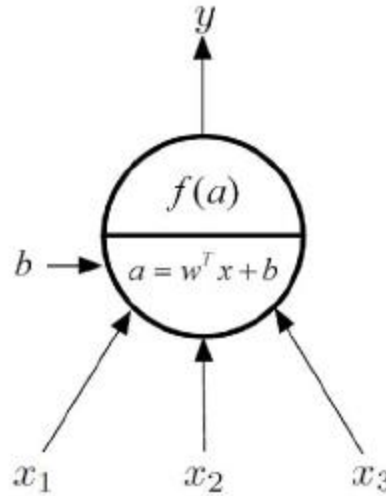
Deep Learning Neural Network



● Input Layer ● Hidden Layer ● Output Layer

- ▶ Based on the observed behaviour of biological neurons.
- ▶ Used to mimic the functioning of our brain.
- ▶ Neural Network is just a group of different neurons strong together
- ▶ Group of Neurons are divided into layers

Neuron Model



- ▶ Input : a vector X & bias term b
- ▶ First inputs are linearly aggregated $a = x_1w_1 + x_2w_2 + x_3w_3 + b$
- ▶ Output : $y = f(a)$
- ▶ Limitation of single neurons : A decision border of a single sigmoid neuron is a straight line.

Why Multipler Layers ?

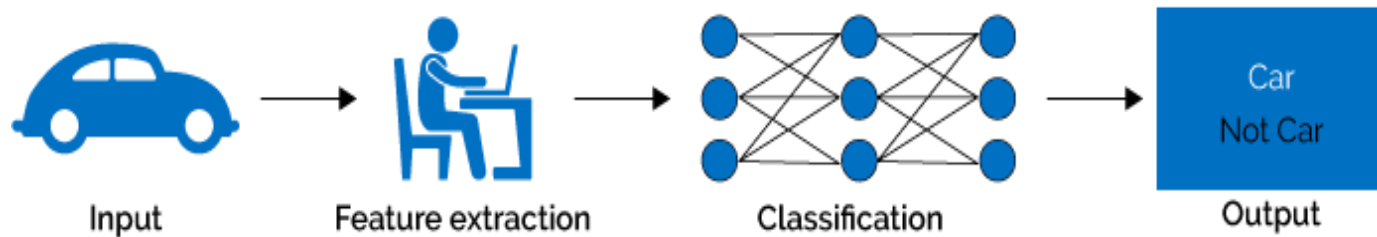
- ▶ **Hierarchical features**
- ▶ To learn **more detailed and more abstractions relationships** within the data and **how the features interact with each other** on a non-linear level.
- ▶ **By adding more hidden layers / more neurons per layer which adds more parameters to the model**
- ▶ Allow the model to **fit more complex functions**
- ▶ **Small Neural Network → Prone to Under fitting**
(Computationally Cheaper)
- ▶ **Large Neural Network → (more Parameters) Prone to Over fitting**
(Computationally expensive)

Object Detection → Classify object in Image

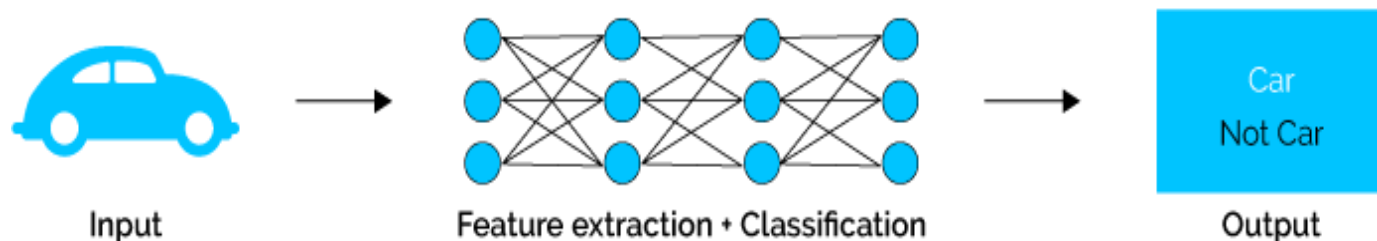
- ▶ One thinks of each layer as an abstraction level
- ▶ To learn **more detailed and more abstractions relationships** within the data and **how the features interact with each other** on a non-linear level.
- ▶ **By adding more hidden layers / more neurons per layer which adds more parameters to the model**
- ▶ Allow the model to **fit more complex functions**
- ▶ **Small Neural Network → Prone to Under fitting (Computationally Cheaper)**
- ▶ **Large Neural Network → (more Parameters) Prone to Over fitting & Computationally expensive**

Difference between DL & ML ?

Machine Learning



Deep Learning



Difference between ML& DL ?

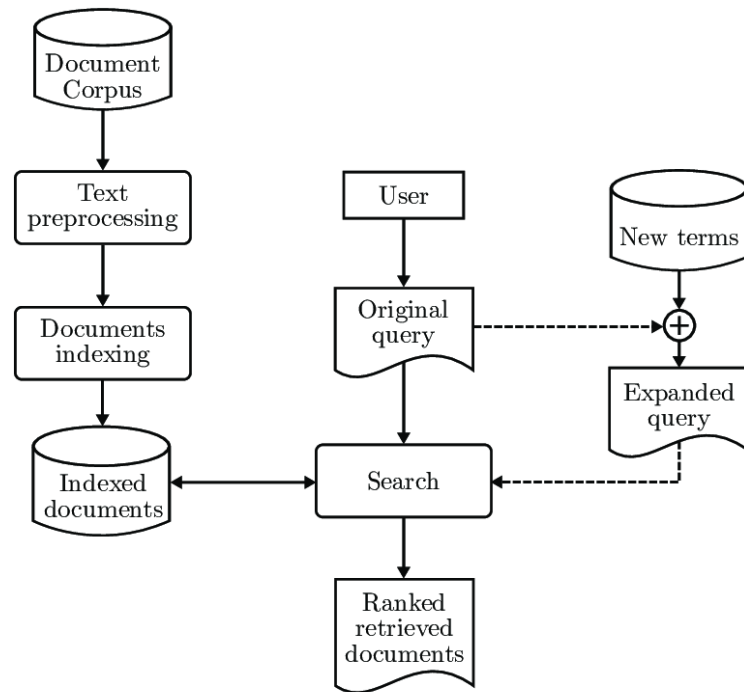
Machine Learning

- 1) Take Decision based on past data
- 2) Need small amount of data
- 3) Work well on low end systems (relatively required low computation power)
- 4) Most feature need to be identified in advance and manually coded.
- 5) Problems is divided into parts and solved individually
- 6) Testing time longer

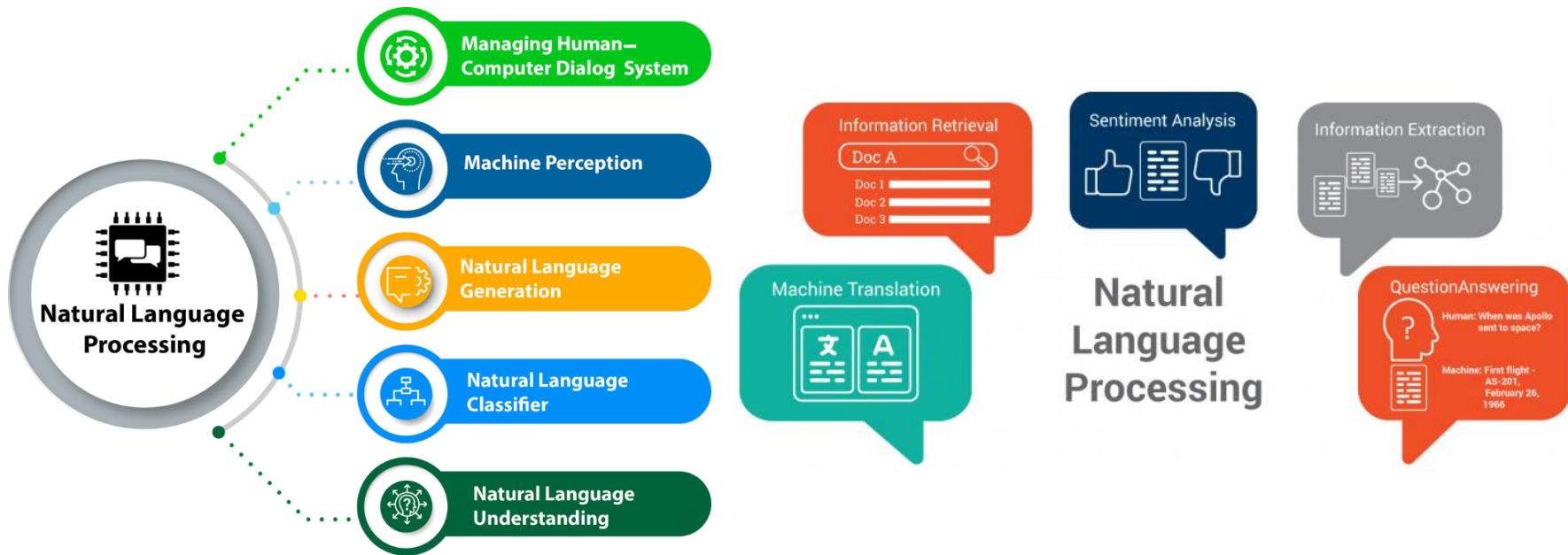
Deep Learning

- 1) Take Decision with the help of artificial neural network
- 2) Need (loads) large amount of data
- 3) Need high end system (need high computation power)
- 4) Learn features from the data it is provided
- 5) The problem is solved end to end within multiple layers
- 6) Testing takes less time comparatively

Information Retrieval



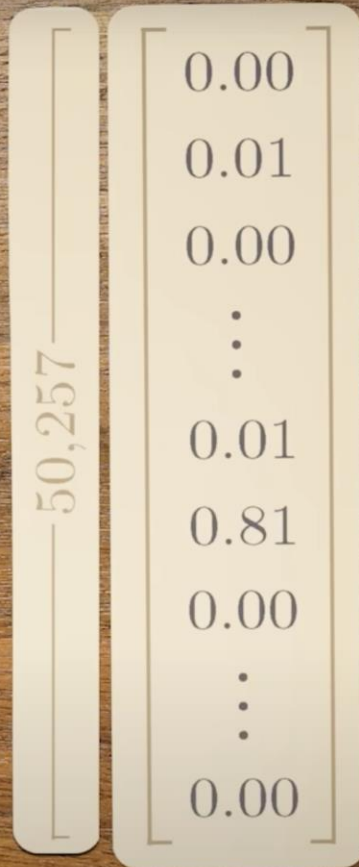
Natural Language Processing



Einstein's first name is



LLM



!

"

#

accompanied

Albert (42,590)

Mongo

<|endoftext|>

$$P(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Softmax

$$\frac{+}{1.00}$$

ENTROPY OF NATURAL LANGUAGE

A neural network is a **method** in artificial intelligence that teaches computers to process data...

aws.amazon.com/what-is/neural-network

A neural network is a **machine** learning program, or model, that makes decisions in a manner...

ibm.com/topics/neural-networks

A neural network is a **group** of interconnected units called neurons that send signals to one another...

wikipedia.org/wiki/Neural_network

A neural network is a **series** of algorithms that endeavors to recognize...

investopedia.com/terms/n/neuralnetwork.asp

A neural network is a **type** of machine learning algorithm inspired by the human brain...

cloud.google.com/discover/what-is-a-neural-network

A neural network is a **computing** model whose layered structure...

databricks.com/glossary/neural-network

A neural network is a **bunch** of inputs and outputs...

aimi.com/insights/what-is-a-neural-network

GPT-4 Technical Report

OpenAI*

Abstract

We report the development of GPT-4, a large-scale, multimodal model which can accept image and text inputs and produce text outputs. While less capable than humans in many real-world scenarios, GPT-4 exhibits human-level performance on various professional and academic benchmarks, including passing a simulated bar exam with a score around the top 10% of test takers. GPT-4 is a Transformer-based model pre-trained to predict the next token in a document. The post-training alignment process results in improved performance on measures of factuality and adherence to desired behavior. A core component of this project was developing infrastructure and optimization methods that behave predictably across a wide range of scales. This allowed us to accurately predict some aspects of GPT-4's performance based on models trained with no more than 1/1,000th the compute of GPT-4.

<https://arxiv.org/pdf/2303.08774>

2 Scope and Limitations of this Technical Report

This report focuses on the capabilities, limitations, and safety properties of GPT-4. GPT-4 is a Transformer-style model [39] pre-trained to predict the next token in a document, using both publicly available data (such as internet data) and data licensed from third-party providers. The model was then fine-tuned using Reinforcement Learning from Human Feedback (RLHF) [40]. Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar.

We are committed to independent auditing of our technologies, and shared some initial steps and ideas in this area in the system card accompanying this release.² We plan to make further technical details available to additional third parties who can advise us on how to weigh the competitive and safety considerations above against the scientific value of further transparency.

Generative AI Overview – GTP 4

Can ChatGPT exist without RLHF ?

RLHF seems to be necessary, but not sufficient

RLHF is a key factor in many popular models, both on and off the record, including ChatGPT, Gemini, Claude, Llama 2, and more

A heavily abbreviated history of LMs

1948: Claude Shannon models English

1948-2017: 🍷

2017: the transformer is born

2018: GPT-1, ELMo and BERT released

2019: GPT-2 and scaling laws

2020: GPT-3 surprising capabilities

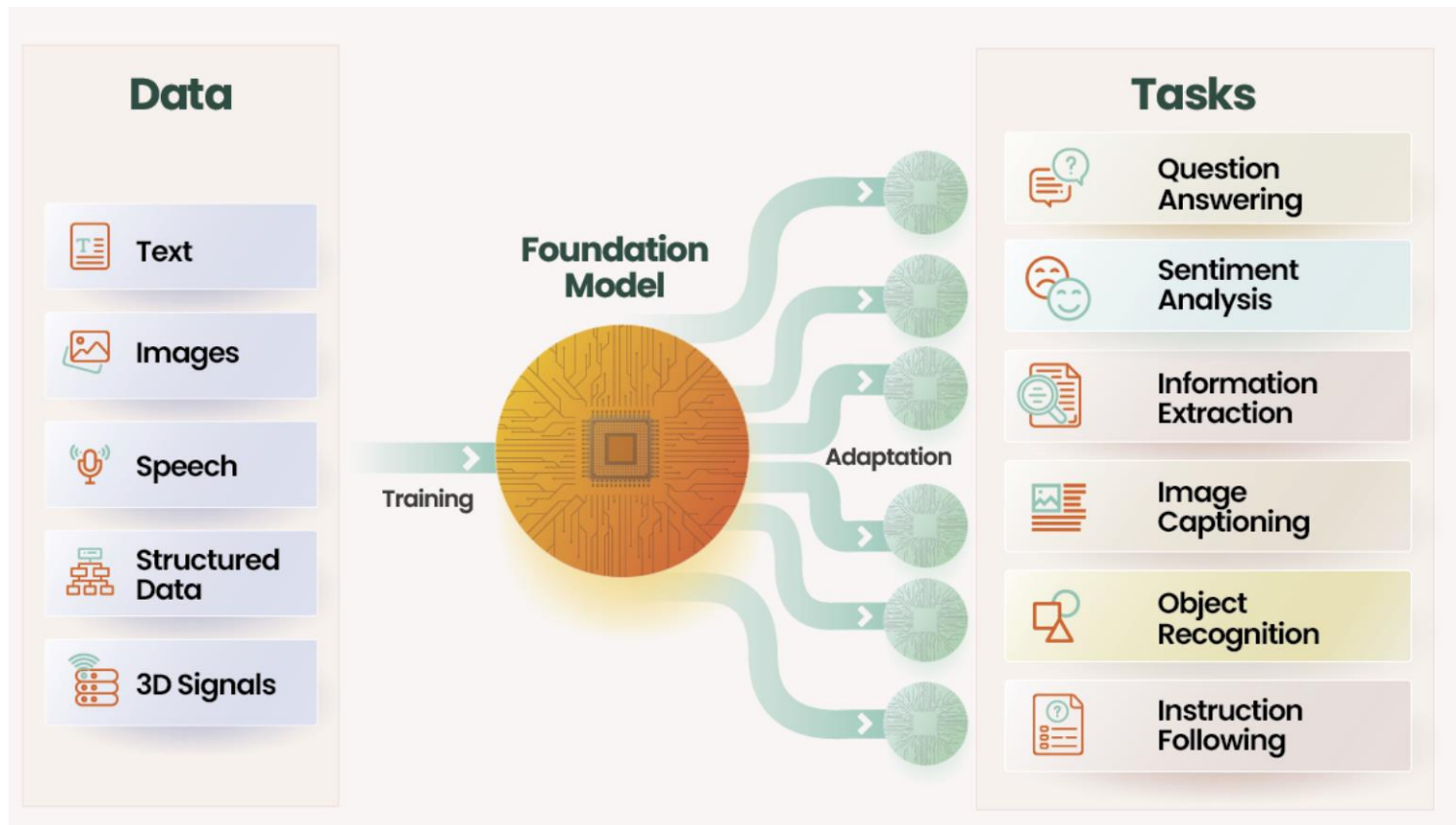
2021: Stochastic parrots

2022: **ChatGPT**

“Meanwhile reinforcement learning, known for its instability, seemed a somewhat shadowy field for those in the NLP research community. However, reinforcement learning proved highly effective, particularly given its cost and time effectiveness.”

- Touvron, H. et al. “ Llama 2: Open Foundation and Fine-Tuned Chat Models.” 2023

Generative AI Overview



LLM Tokenization – LLM Prompt Request Processing

Tiktokenizer

System ▾ You are a helpful assistant ✕

User ▾ Content ✕

Add message

मराठी भाषा ही इंडो युरोपीय भाषाकुळातील एक भाषा आहे

```
public static void main(String [] args){  
  int x = 20;  
  int y = 30;  
  int z = x + y;  
}
```

explain the code for me

gpt-4o ▾

Token count
73

मराठी भाषा ही इंडो युरोपीय भाषाकुळातील एक भाषा आहे

```
public static void main(String [] args){  
  int x = 20;  
  int y = 30;  
  int z = x + y;  
}
```

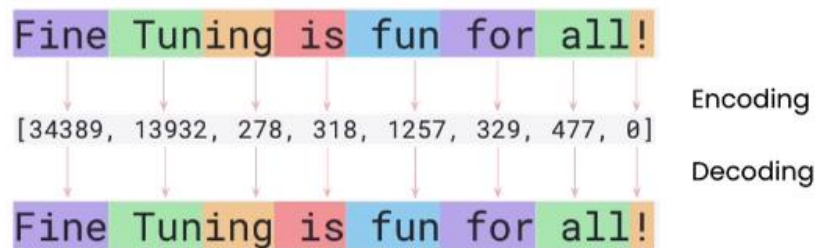
explain the code for me

135854, 50307, 108881, 12010, 71377, 952, 4254, 8929, 952, 28498, 2762, 70625, 9384, 89921, 98446, 6298, 108881, 22278, 279, 1154, 1633, 1010, 2758, 2930, 6126, 4966, 2138, 491, 159732, 227, 87, 159732, 227, 28, 159732, 227, 455, 307, 491, 159732, 227, 88, 159732, 227, 28, 159732, 227, 1130, 307, 491, 159732, 227, 89, 159732, 227, 28, 159732, 227, 87, 159732, 227, 10, 159732, 227, 88, 307, 943, 490, 26330, 290, 3490, 395, 668

☐ Show whitespace

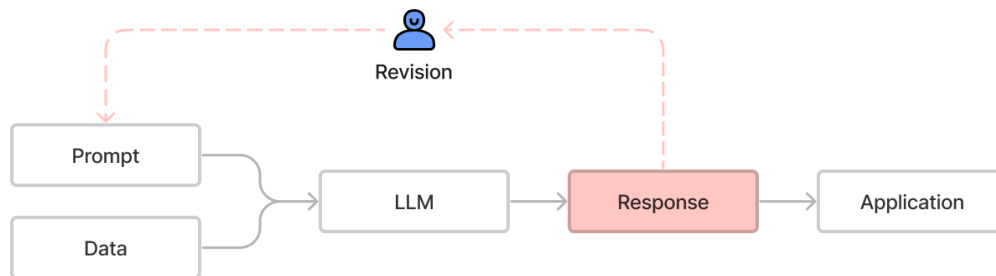
LLM Tokenization – LLM Prompt Request Processing

- Tokenize the data

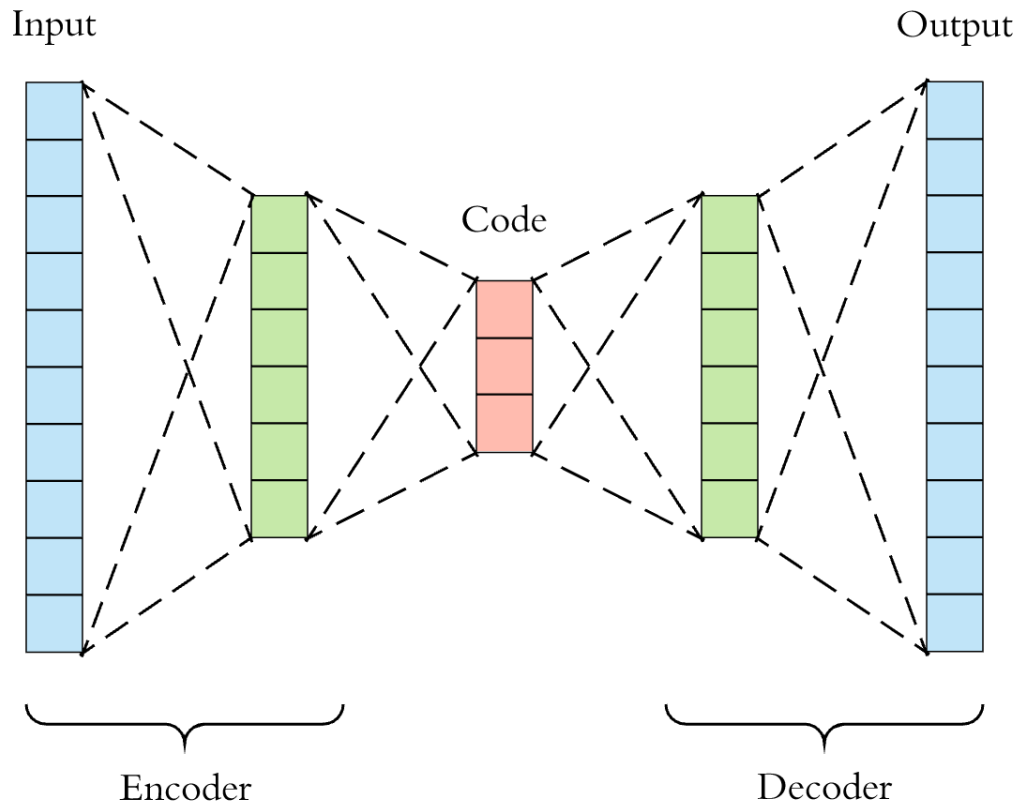


There are multiple popular tokenizers:

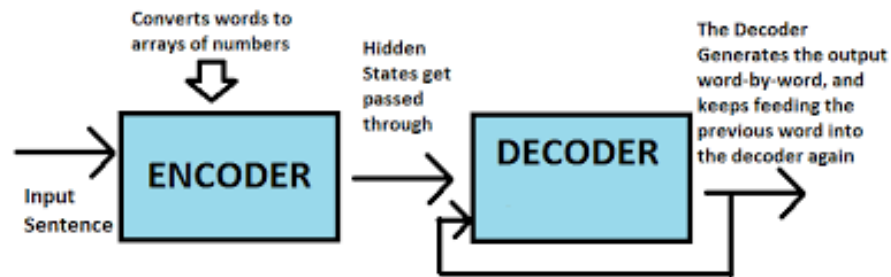
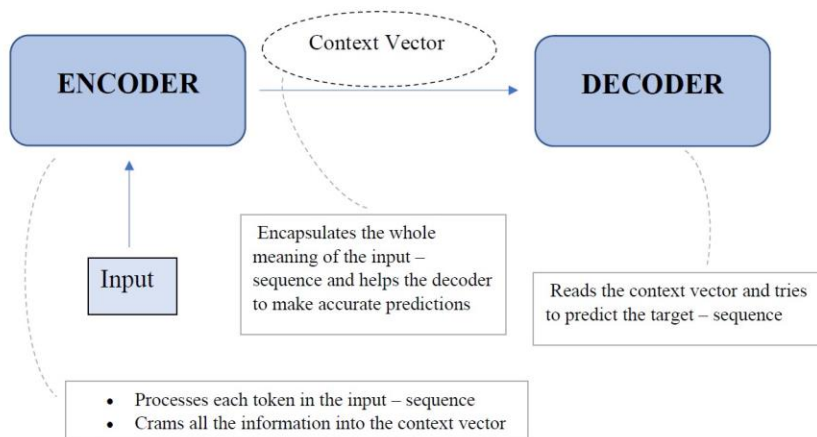
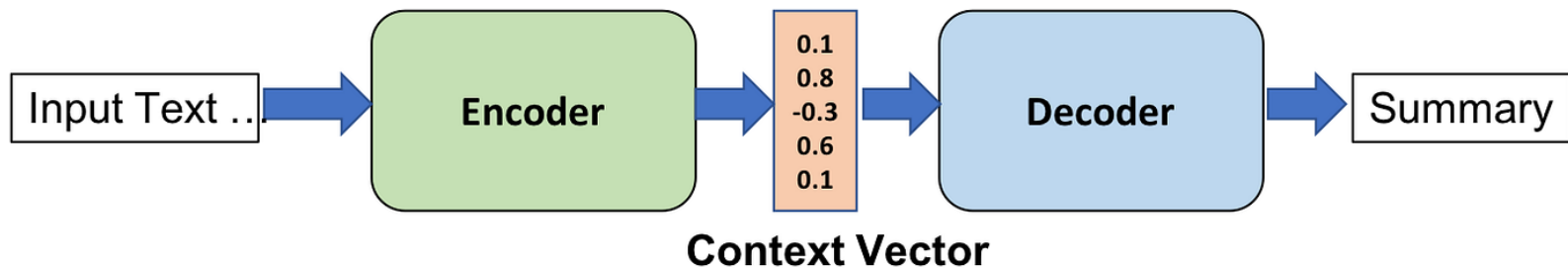
- Use the tokenizer associated with your model!



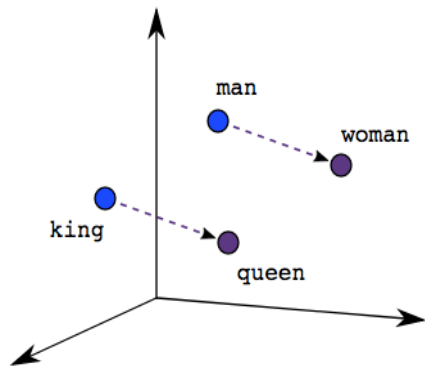
Encoder and Decoder



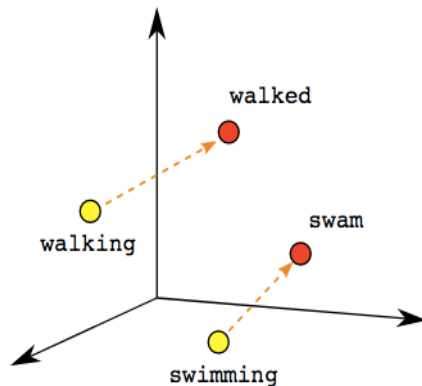
Encoder and Decoder



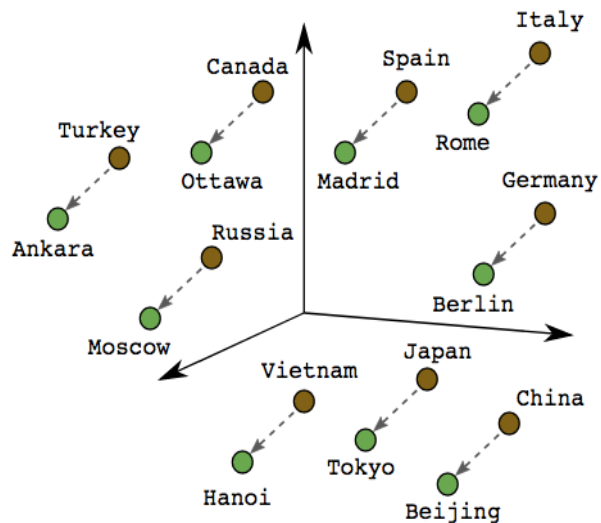
Word Embedding



Male-Female

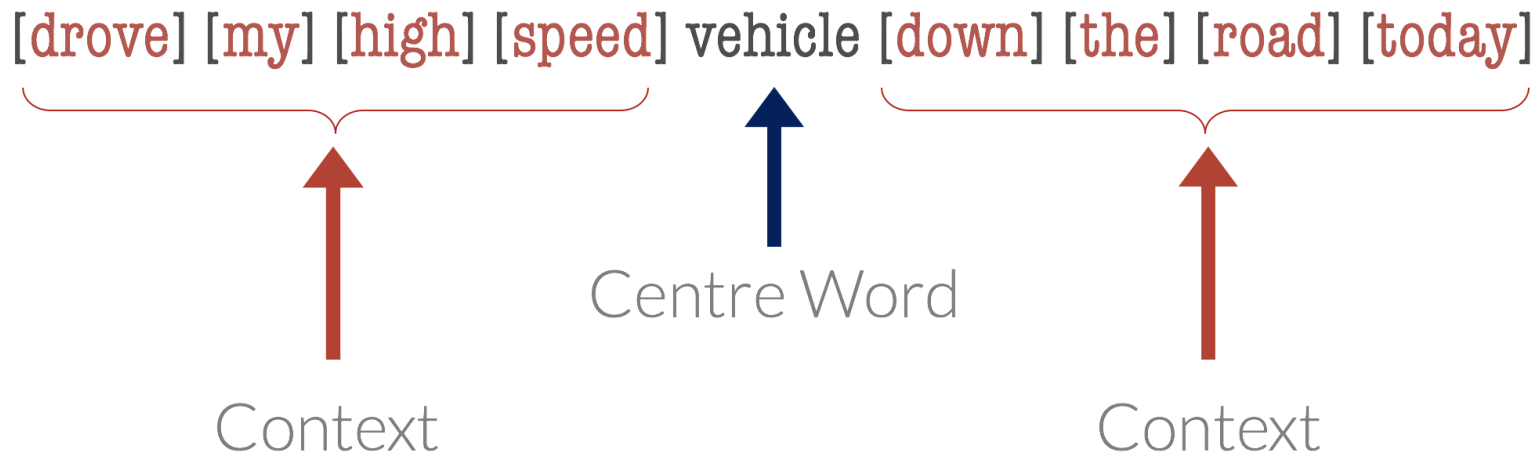


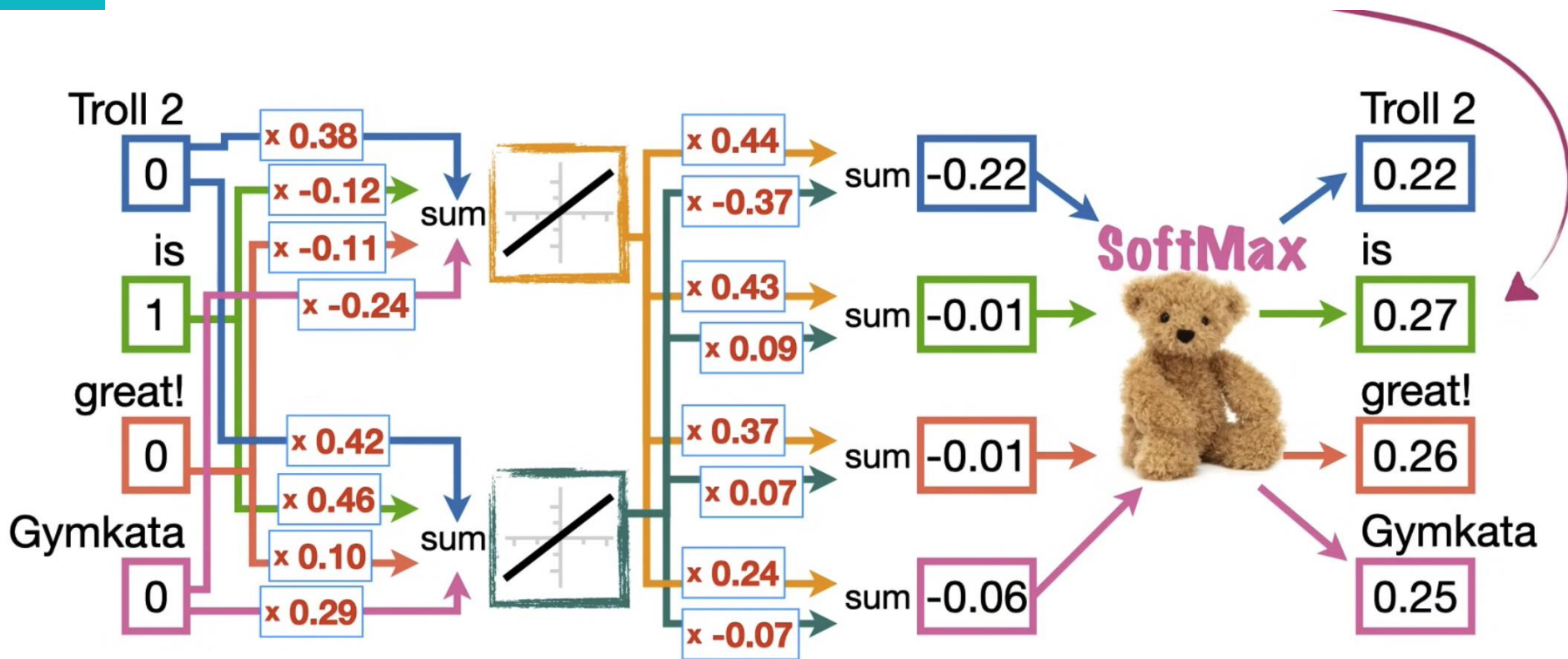
Verb Tense



Country-Capital

Word Embedding and Capturing Context





Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions

Transformer

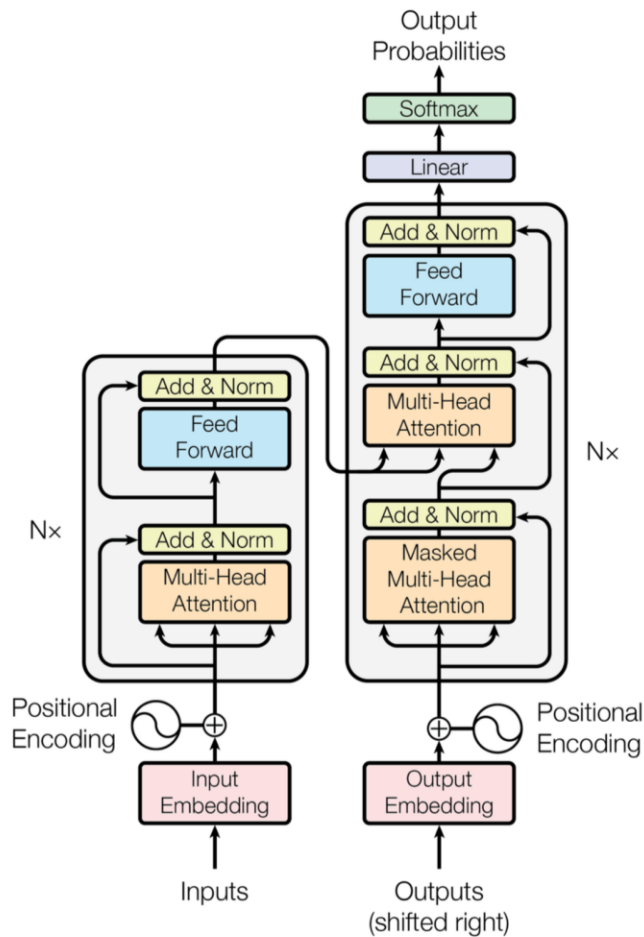
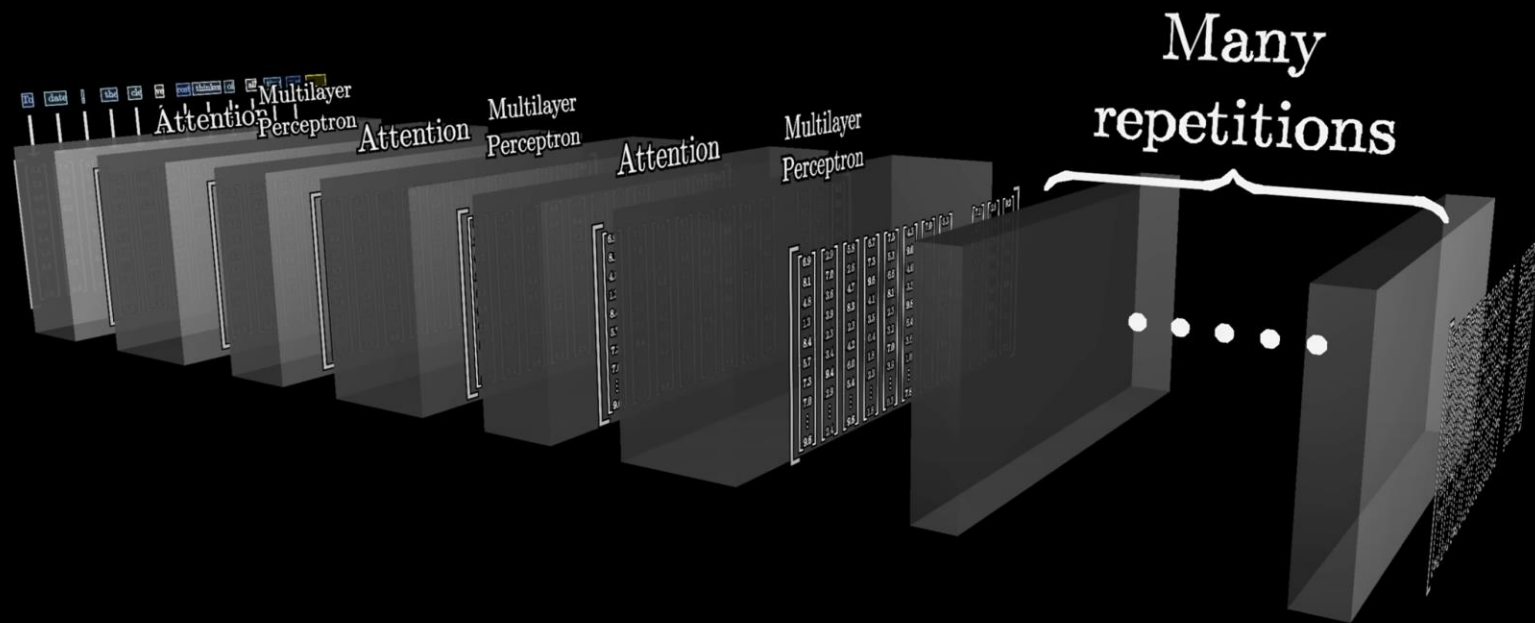
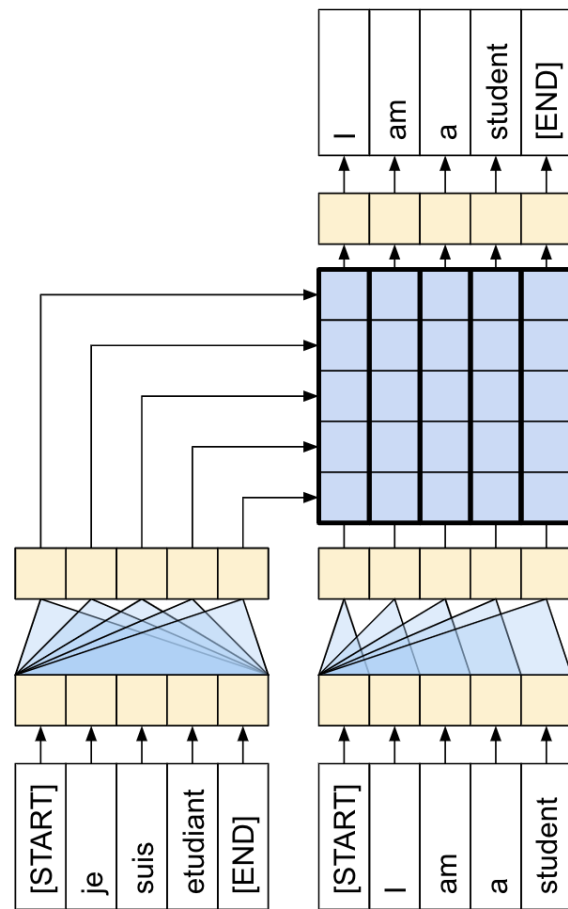
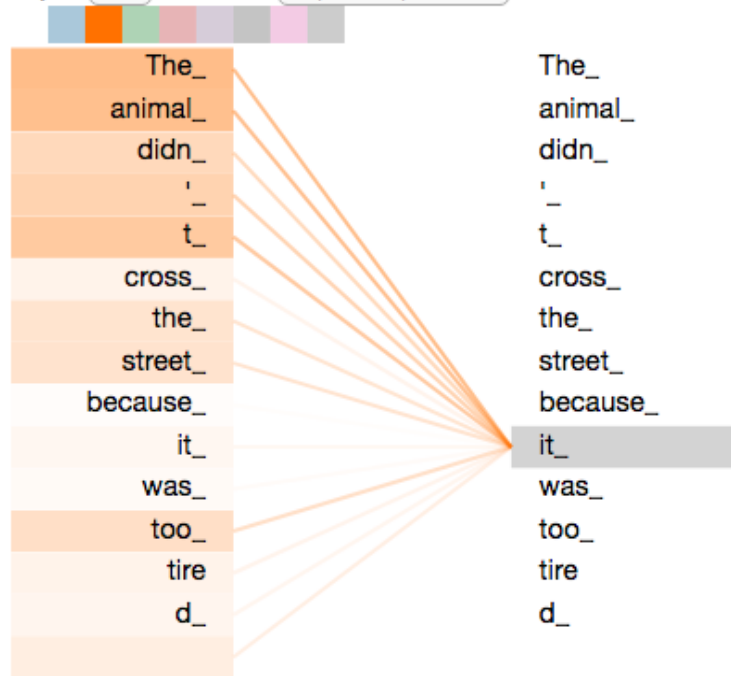


Figure 1: The Transformer - model architecture.



Transformer

Layer: 5 Attention: Input - Input



Transformer

1. Attention Mechanism

- ▶ Imagine you're reading a sentence: "The cat sat on the mat, which was blue."
- ▶ **Older models:** Might process this word-by-word, potentially losing the connection between "mat" and "blue."
- ▶ **Transformer with attention:** Can see the whole sentence at once. When processing "blue," it pays strong attention to "mat" and "which," understanding that the "blue" refers to the "mat."
- ▶ **Example:** In a translation task, attention helps accurately translate pronouns by focusing on the relevant noun they refer to, even if it's far away in the sentence.

2. Encoder-Decoder Structure

Think of translating English to French:

- ▶ **Encoder:** Reads the English sentence ("I love to eat pizza") and creates a rich representation capturing its meaning.
- ▶ **Decoder:** Takes this representation and generates the French translation ("J'aime manger de la pizza").

Example: In a chatbot, the encoder understands your question, and the decoder generates a relevant response.

3. **Self-Attention** - The Core Idea - The Heart of Transformers

Consider the sentence: "The bird flew over the mountain."

- ▶ **For the word "flew":** Self-attention allows it to focus on "bird" (who flew?) and "mountain" (flew over what?), capturing the relationships and understanding the action.

Example: In sentiment analysis, self-attention helps identify how words like "not" or "very" modify the sentiment of other words in the sentence.

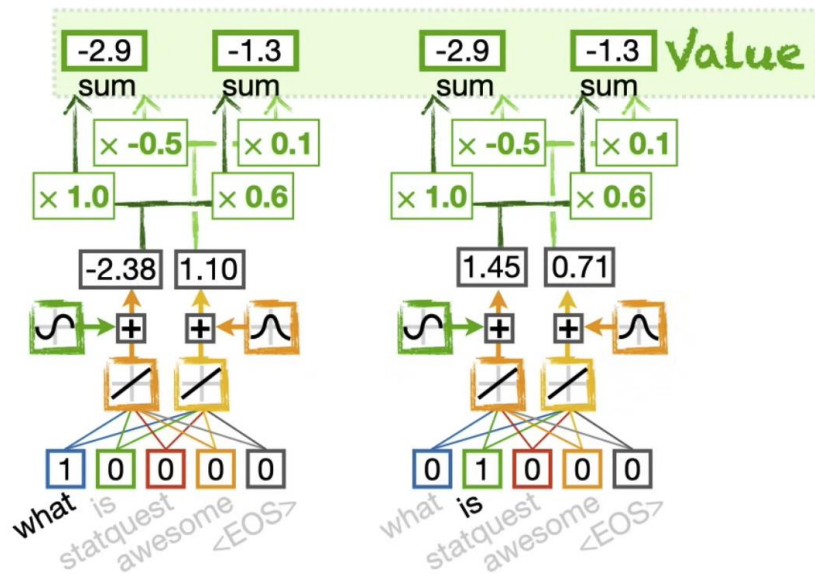
Transformer – self attention

Masked Self-Attention values
for **what**

-2.9 -1.3

Masked Self-Attention values
for **is**

1.1 0.9



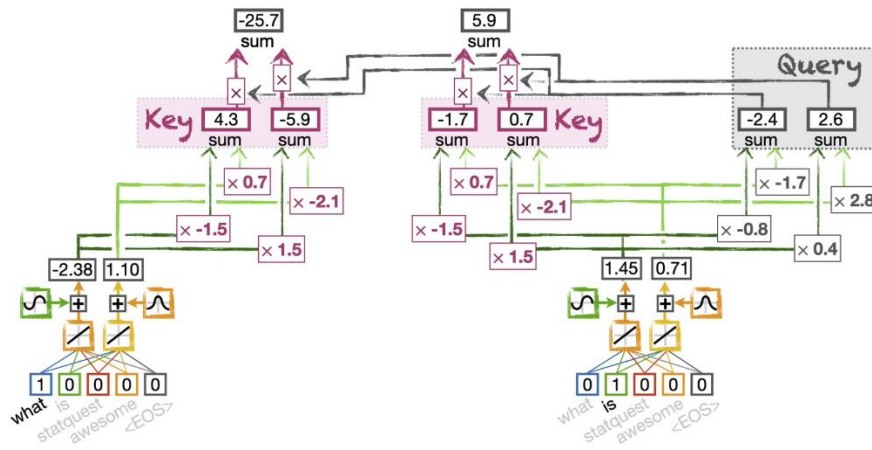
Transformer

4. Multi-Head Attention – Build Diverse Perspectives

Imagine analyzing the sentence: "She threw the ball to him."

- ▶ **One attention head:** Might focus on the action ("threw").
- ▶ **Another head:** Might focus on the participants ("she," "him").
- ▶ **Yet another head:** Might focus on the direction ("to").

Example: In question answering, different attention heads might focus on the question, the context, and the potential answer within the text.



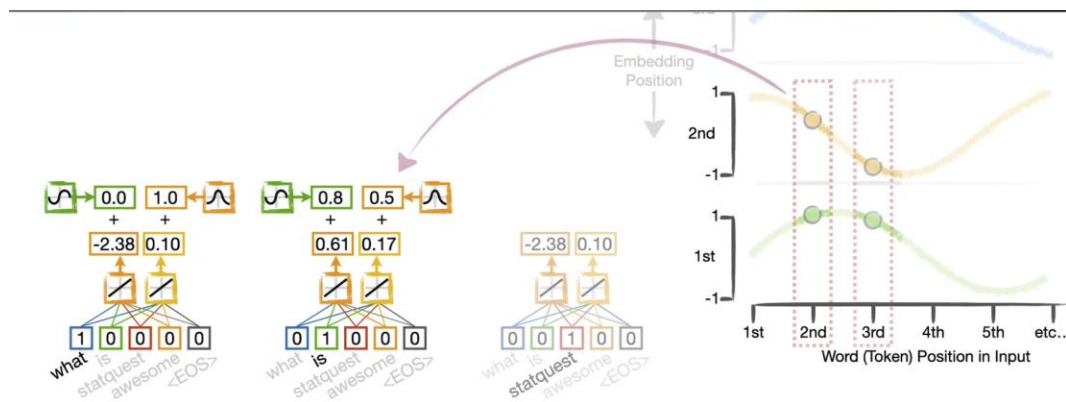
Transformer

5. Positional Encoding - Maintaining Order

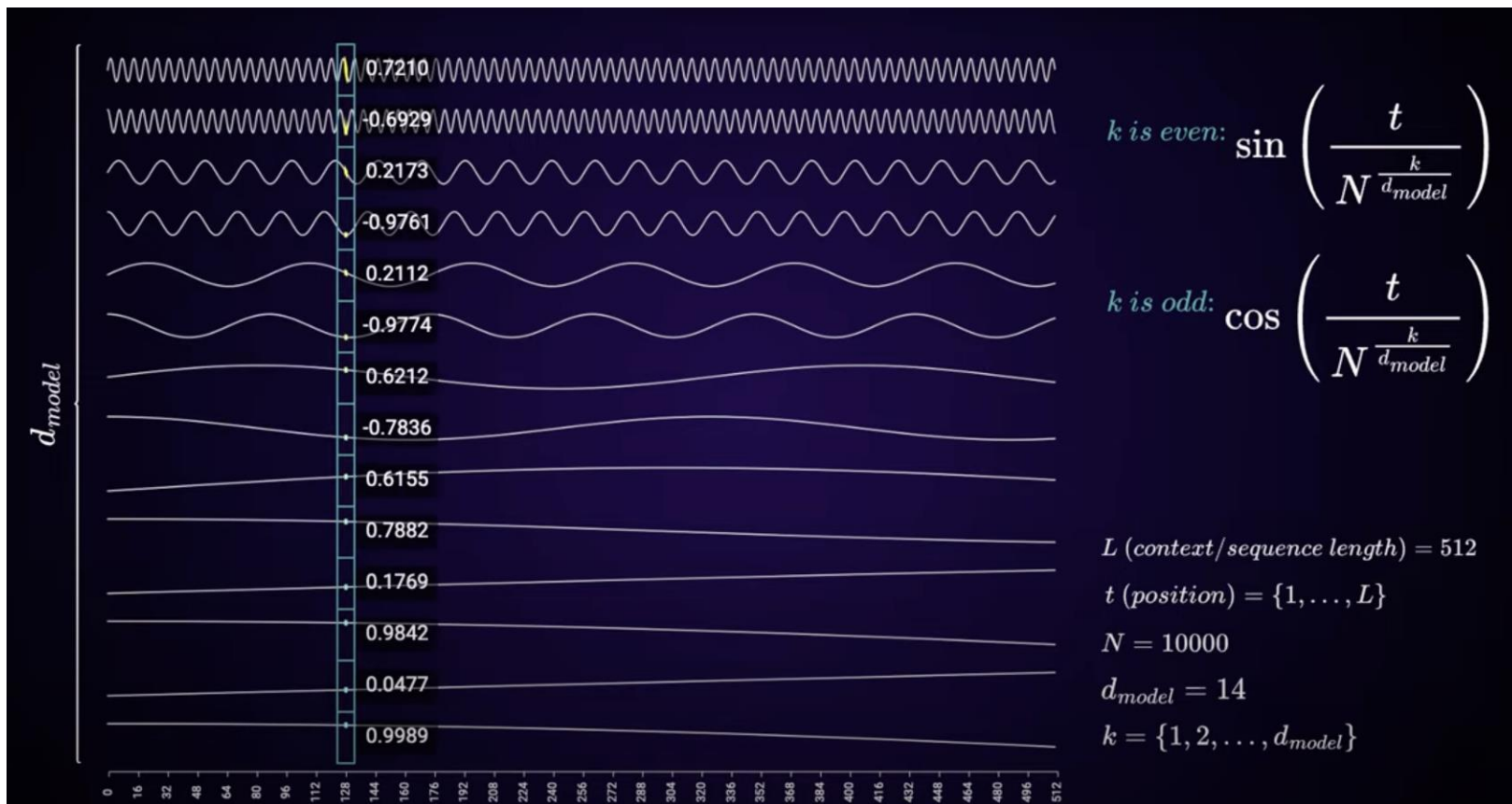
Consider the sentences: "The dog bit the man" vs. "The man bit the dog."

- ▶ **Without positional encoding:** The model might see both as the same.
- ▶ **With positional encoding:** It understands the different word order and thus the different meanings.

Example: In summarizing a news article, positional encoding helps the model maintain the chronological order of events.



Transformer – Positional Encoding



6. Residual Connections and Layer Normalization - Enhancing Training

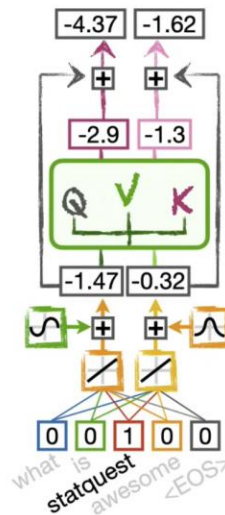
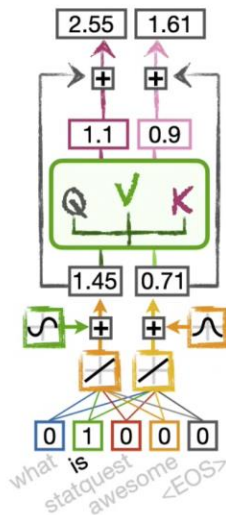
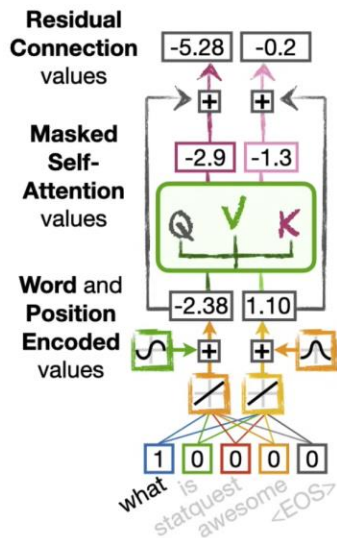
Each sub-layer in the encoder and decoder uses residual connections and layer normalization.

These techniques help to stabilize training and improve the flow of gradients during backpropagation, enabling the model to learn more effectively.

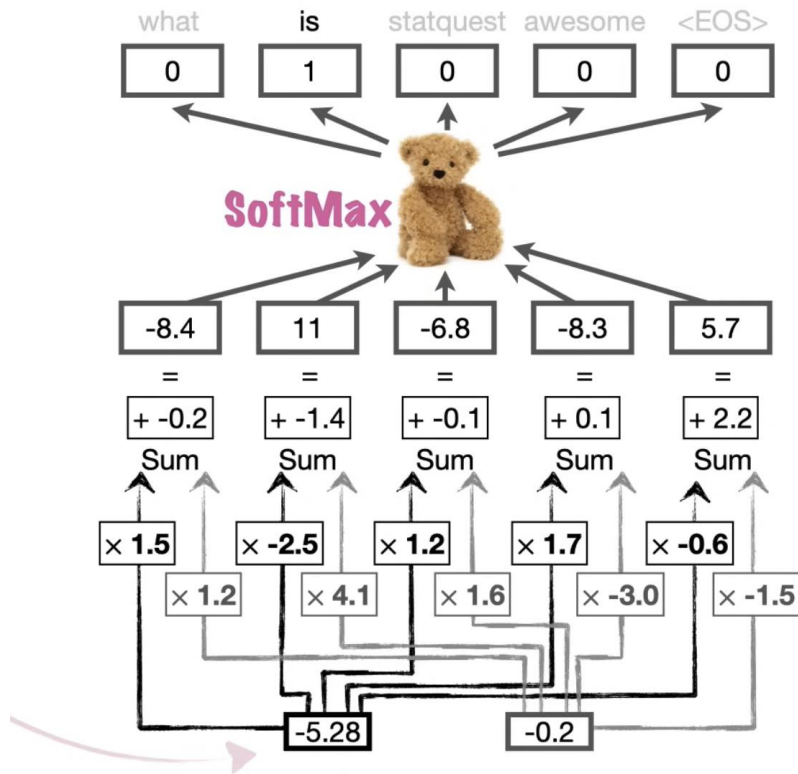
The Transformer architecture has spawned a whole family of powerful language models, including:

- ▶ **GPT (Generative Pre-trained Transformer):** Excels at text generation tasks.

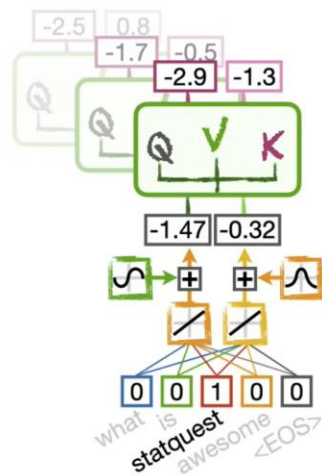
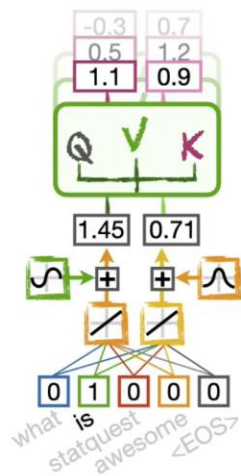
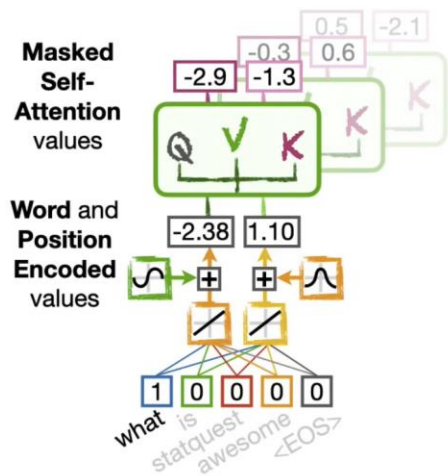
Bert- Stacked Encoder Transformer



GTP – Stacked Decoder Transformer



GTP – Stacked Decoder Transformer

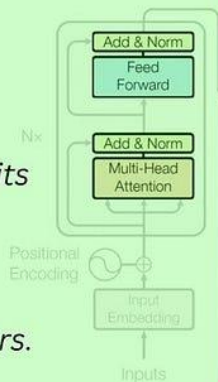


Bert Vs GPT

BERT

Google

use transfer learning to **continue learning** from its existing data when adding user-specific tasks and layers.



GPT

OpenAI

decodes from its massive pre-learned embeddings to present output that matches user prompts. It

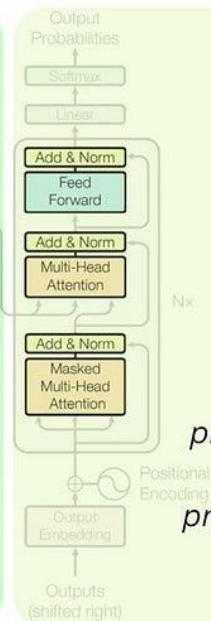
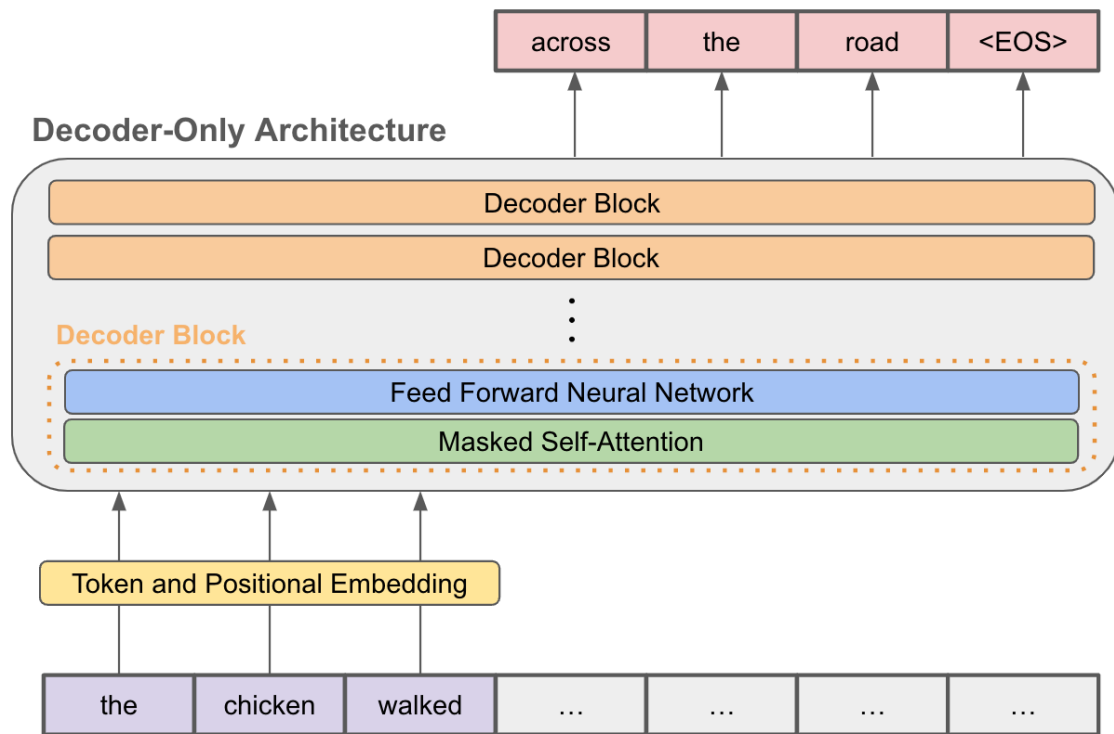
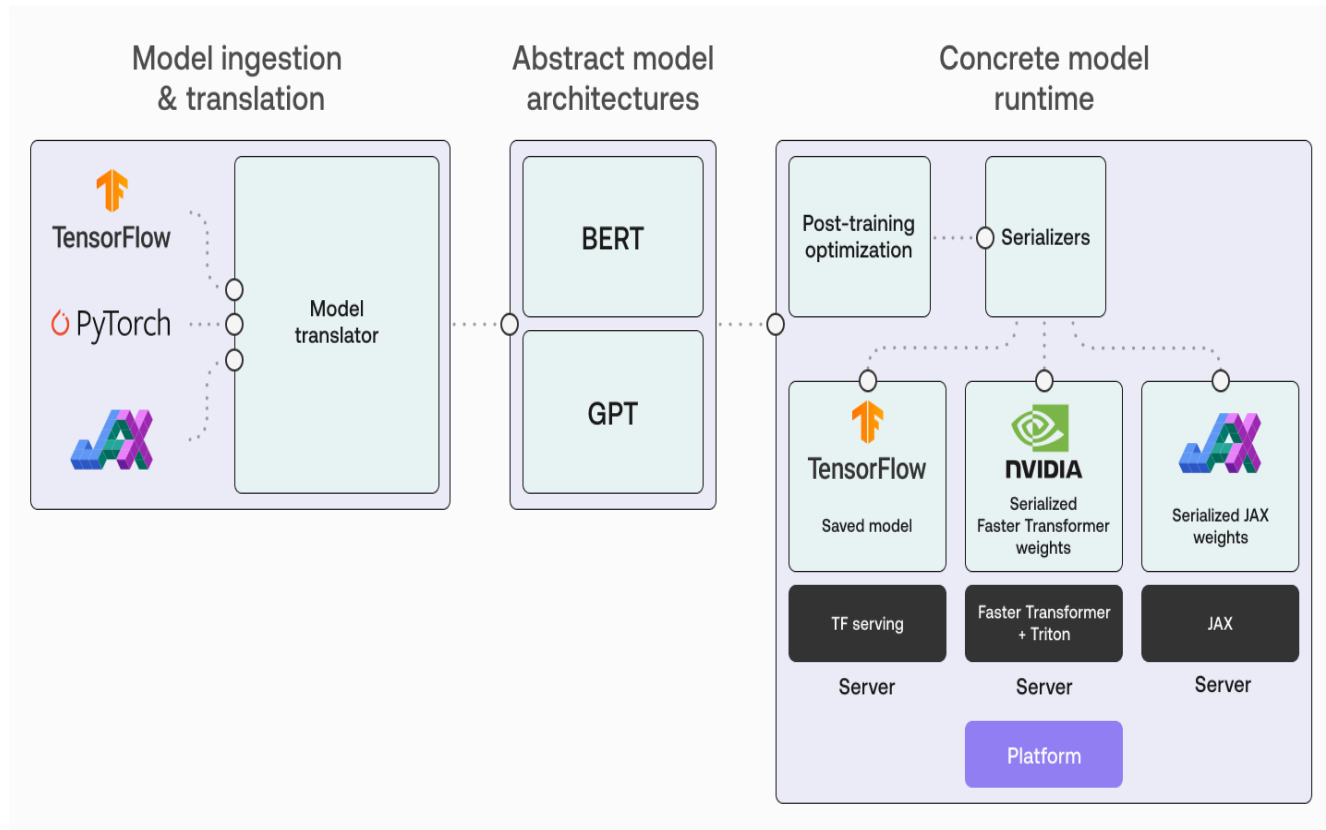


Figure 1: The Transformer - model architecture.

Gemini and ChatGPT – Request Response using Prompt



Generative AI



Thank You